

EXERCISE 11.1

Write monad instances for `Par`, `Parser`, `Option`, `Stream`, and `List`.

EXERCISE 11.2

Hard: `State` looks like it would be a monad too, but it takes two type arguments and you need a type constructor of one argument to implement `Monad`. Try to implement a `State` monad, see what issues you run into, and think about possible solutions. We'll discuss the solution later in this chapter.

11.3 Monadic combinators

Now that we have our primitives for monads, we can look back at previous chapters and see if there were some other functions that we implemented for each of our monadic data types. Many of them can be implemented once for all monads, so let's do that now.

EXERCISE 11.3

The sequence and traverse combinators should be pretty familiar to you by now, and your implementations of them from various prior chapters are probably all very similar. Implement them once and for all on `Monad[F]`.

```
def sequence[A](lma: List[F[A]]): F[List[A]]
def traverse[A,B](la: List[A])(f: A => F[B]): F[List[B]]
```

One combinator we saw for `Gen` and `Parser` was `listOfN`, which allowed us to replicate a parser or generator `n` times to get a parser or generator of lists of that length. We can implement this combinator for all monads `F` by adding it to our `Monad` trait. We should also give it a more generic name such as `replicateM` (meaning “replicate in a monad”).

EXERCISE 11.4

Implement `replicateM`.

```
def replicateM[A](n: Int, ma: F[A]): F[List[A]]
```